Deep Research: Avatrada Ui 9 Topic 005

Engineering Report: Algorithmic Trading Execution Engine

To: Lead Architect, Proprietary Trading Platform **From:** Quantitative Development & Research **Date:** October 26, 2023 **Subject:** Technical Deep-Dive on Core Execution Logic Components

1. Client-Side Bracket Order State Machine

1.1. Technical Deconstruction

A Bracket Order is a composite order structure designed to automate trade management by linking an entry order with two exit orders: a Take Profit (Limit Order) and a Stop Loss (Stop Order). These two exit orders are placed in a One-Cancels-Other (OCO) group. The entire lifecycle is managed by a state machine that reacts to events from the brokerage API (e.g., order submissions, fills, rejections).

The core logic flow is as follows: 1. The parent (entry) order is submitted. 2. The state machine waits for a fill confirmation for the parent order. 3. Upon a full fill of the parent order, the state machine immediately submits the two child orders (Take Profit and Stop Loss) as an OCO group. 4. If one of the child orders fills, the state machine sends a cancellation request for the other, remaining child order. 5. The lifecycle completes when the position is closed and all associated orders are terminated.

1.2. Implementation Strategy

The system can be implemented as an object-oriented state machine. A `BracketOrderController` class would manage the state and the associated order objects.

States: * `PENDING_SUBMISSION`: The initial state before the entry order is sent to the broker. * `ENTRY_SUBMITTED`: The entry order has been sent and we are awaiting a response. * `PARTIALLY_FILLED`: The entry order has received one or more partial fills but is not yet complete. * `ENTRY_FILLED`: The entry order is fully filled. This is the trigger state for submitting the OCO children. * `OCO_ACTIVE`: The Take Profit and Stop Loss orders have been successfully submitted and are working in the market. * `COMPLETED`: One of the OCO children has filled, the other has been successfully cancelled, and the trade is closed. * `CANCELLED`: The bracket order was cancelled by the user before completion. * `REJECTED`: The entry order or one of the child orders was rejected by the exchange/broker.

State Transition Logic (Simplified):

```
(Start) -> PENDING_SUBMISSION
    |
    | user.submit()
    V
ENTRY_SUBMITTED --(broker.reject)--> REJECTED
    |
    | broker.partial_fill()
    V
PARTIALLY_FILLED --(broker.fill)--> ENTRY_FILLED
    | ^
    | | broker.partial_fill()
    |
    | broker.full_fill()
    V
ENTRY_FILLED
    |
    | system.submit_oco()
    V
OCO_ACTIVE --(broker.reject_oco)--> REJECTED
    |
```

```
    | broker.child_fill()
    V
COMPLETED
```

Pseudocode for the Controller:

```
class BracketOrderController:
    def __init__(self, entry_order, tp_order, sl_order):
        self.state = "PENDING_SUBMISSION"
        self.entry_order = entry_order
        self.tp_order = tp_order
        self.sl_order = sl_order
        self.entry_order.parent_controller = self
        # Child orders are not submitted yet

    def submit_entry(self):
        if self.state == "PENDING_SUBMISSION":
            # api.submit(self.entry_order) -> returns order_id
            self.entry_order.id = api.submit(self.entry_order)
            self.state = "ENTRY_SUBMITTED"
            print(f"State -> ENTRY_SUBMITTED for Order {self.entry_order.id}")

    def on_broker_event(self, event):
        # This method is called by a central event listener
        if event.order_id != self.entry_order.id and event.order_id not in
[self.tp_order.id, self.sl_order.id]:
            return # Not our event

        # --- Entry Order Logic ---
        if self.state == "ENTRY_SUBMITTED" or self.state == "PARTIALLY_FILLED":
            if event.type == "FILL" and event.remaining_qty == 0:
                self.state = "ENTRY_FILLED"
                print(f"State -> ENTRY_FILLED for Order {self.entry_order.id}")
                self.submit_oco_children()
            elif event.type == "FILL" and event.remaining_qty > 0:
                self.state = "PARTIALLY_FILLED"
                print(f"State -> PARTIALLY_FILLED for Order
{self.entry_order.id}")
            elif event.type == "REJECT":
```

```
                self.state = "REJECTED"
                print(f"State -> REJECTED for Order {self.entry_order.id}")

        # --- OCO Logic ---
        elif self.state == "OCO_ACTIVE":
            if event.type == "FILL":
                if event.order_id == self.tp_order.id:
                    api.cancel(self.sl_order.id)
                    self.state = "COMPLETED"
                    print(f"State -> COMPLETED. TP Filled, SL Cancelled.")
                elif event.order_id == self.sl_order.id:
                    api.cancel(self.tp_order.id)
                    self.state = "COMPLETED"
                    print(f"State -> COMPLETED. SL Filled, TP Cancelled.")

    def submit_oco_children(self):
        # Assumes broker API supports OCO groups. If not, this logic becomes
more complex.
        self.tp_order.id, self.sl_order.id = api.submit_oco(self.tp_order,
self.sl_order)
        if self.tp_order.id and self.sl_order.id:
            self.state = "OCO_ACTIVE"
            print(f"State -> OCO_ACTIVE. TP={self.tp_order.id},
SL={self.sl_order.id}")
        else:
            self.state = "REJECTED" # Or some error state
            print("ERROR: OCO submission failed.")
```

1.3. Critical Analysis

- **Latency Risk:** There is an inherent latency gap between the parent order fill notification and the submission of the child OCO orders. In a fast-moving market, the price could move through the Stop Loss level before the order is even placed, leading to greater-than-expected losses. This is a fundamental risk of client-side bracket orders.
- **Partial Fill Complexity:** The provided logic waits for a full fill. A more robust implementation must decide how to handle partial fills. Options include: a) submitting a proportionally sized bracket for the filled quantity,

or b) waiting for the full fill. Strategy (a) adds significant complexity to state management.

- **API Failure Modes:** The cancellation request for the remaining OCO order could fail (e.g., network error, API downtime). This would leave a "dangling" order in the market, which could be executed unintentionally. The system requires a robust reconciliation layer to periodically check for and handle such orphaned orders.
- **State Persistence:** If the client application crashes, all in-memory state is lost. For a robust system, the state of every bracket order must be persisted to a database or log file to allow for recovery on restart.

---

2. Pre-Flight Validation Layer

2.1. Technical Deconstruction

The Pre-Flight Validation Layer acts as a critical safety mechanism, intercepting every order request generated by the user or strategy engine before it is submitted to the broker's API. Its purpose is to prevent costly errors by enforcing a set of predefined risk rules locally. This reduces the likelihood of erroneous orders reaching the market and minimizes API rejection traffic.

The required checks are: 1. **Max Position Size:** Ensures the new order will not cause the total position in the instrument to exceed a configured maximum. 2. **Buying Power:** Verifies that the notional value of the order (Price × Quantity) does not exceed the account's available buying power. 3. **Fat Finger Check:** Compares the order's limit price against the last known market price. If the deviation exceeds a threshold (e.g., 5%), the order is rejected to prevent typos.

2.2. Implementation Strategy

This layer can be implemented as a synchronous function or a middleware pattern within the order submission pipeline. It requires access to real-time (or near-real-time) account and market data.

Pseudocode for the Validation Function:

```
 # Data structures assumed to be available
# account_state = { "buying_power": 50000.00, "positions": { "AAPL": 100 } }
# market_data = { "AAPL": { "last_price": 150.00 } }
# risk_config = { "AAPL": { "max_position_size": 500 }, "fat_finger_pct": 0.05 }

def pre_flight_validation(order, account_state, market_data, risk_config):
    """
    Validates an order against pre-defined risk rules.
    Returns a tuple: (is_valid: bool, reason: str)
    """
    symbol = order.symbol

    # --- 1. Max Position Size Check ---
    current_position = account_state.positions.get(symbol, 0)
    max_size = risk_config[symbol].max_position_size
    # Note: This logic assumes long-only. A real system handles long/short
netting.
    if (current_position + order.quantity) > max_size:
        reason = f"VALIDATION FAILED: Order exceeds max position size of
{max_size}."
        return (False, reason)

    # --- 2. Buying Power Check ---
    # This check is most relevant for 'BUY' orders.
    if order.side == "BUY":
        notional_value = order.price * order.quantity
        if notional_value > account_state.buying_power:
            reason = f"VALIDATION FAILED: Notional value ${notional_value}
exceeds buying power ${account_state.buying_power}."
            return (False, reason)

    # --- 3. Fat Finger Check ---
    # This check applies only to orders with a specified price (e.g., Limit,
Stop Limit)
    if hasattr(order, 'price') and order.price is not None:
        last_price = market_data[symbol].last_price
        deviation = abs(order.price - last_price) / last_price

        if deviation > risk_config.fat_finger_pct:
            reason = f"VALIDATION FAILED: Order price ${order.price} deviates
```

```
>{risk_config.fat_finger_pct*100}% from last price ${last_price}."
            return (False, reason)

    # --- All checks passed ---
    return (True, "VALIDATION PASSED")

# --- Example Usage ---
# new_order = Order(symbol="AAPL", quantity=50, price=155.00, side="BUY")
# is_valid, reason = pre_flight_validation(new_order, account_state,
market_data, risk_config)
# if is_valid:
#     api.submit(new_order)
# else:
#     print(reason)
```

2.3. Critical Analysis

- **Stale Data Risk:** The effectiveness of this layer is entirely dependent on the freshness of `account_state` and `market_data`. If the local cache of buying power is stale, the check could erroneously approve an order that the broker will reject. This can happen in a multi-threaded system where another order fills and updates buying power, but the validation layer hasn't received the update yet.
- **Market Order Handling:** The buying power and fat-finger checks as written are insufficient for Market Orders, which have no predefined price. For market orders, the notional value must be estimated using the current ask price (for buys) or bid price (for sells), plus a safety buffer for slippage.
- **Dynamic Thresholds:** A static 5% fat-finger threshold may be too restrictive during high-volatility events (e.g., earnings announcements, news releases) and too loose for stable, low-priced assets. The threshold should be configurable and potentially dynamic, adjusting based on the instrument's recent volatility (e.g., ATR).

---

3. Total Gamma Exposure (GEX) Calculation

3.1. Technical Deconstruction

Gamma Exposure (GEX) is a metric that estimates the total gamma sensitivity of options market makers for a given underlying asset (like an index). It quantifies how much market makers would need to hedge their delta exposure for a $1 move in the underlying. The calculation involves summing the gamma exposure from every options strike in a given chain.

The formula provided in the source context calculates the notional gamma value per strike:

- **GEX per Strike:** `OpenInterest * Gamma * SpotPrice * 100 * (1 for Calls, -1 for Puts)`

This formula calculates the dollar value of the delta that needs to be hedged per 1% move in the underlying. The sign convention treats dealer exposure from sold puts (which requires buying the underlying as it falls) as negative GEX.

3.2. Implementation Strategy

To calculate the **Total Gamma Exposure** for an index, one must iterate through all relevant option strikes for all included expiration dates and sum the GEX per strike.

Mathematical Formula:

Let: * `S` be the Spot Price of the underlying index. * `i` be an index representing each option strike. * `OI_call_i` and `OI_put_i` be the Open Interest for the call and put at strike `i`. * `Γ_call_i` and `Γ_put_i` be the Gamma for the call and put at strike `i`. * `C` be the set of all call strikes to be included. * `P` be the set of all put strikes to be included.

The Total GEX is the summation:

```
Total GEX = (S * 100) * [ Σ_{i ∈ C} (OI_call_i * Γ_call_i) - Σ_{i ∈ P} (OI_put_i * Γ_put_i) ]
```

This can be implemented as a simple loop over the options chain data provided by a data vendor.

Pseudocode for Calculation:

```
def calculate_total_gex(options_chain, spot_price):
    """
    Calculates Total Gamma Exposure from a given options chain.

    options_chain: A list of option contracts, each with attributes like
                   'type', 'strike', 'open_interest', 'gamma'.
    spot_price: The current price of the underlying asset.
    """

    total_call_gex_per_point = 0.0
    total_put_gex_per_point = 0.0

    for contract in options_chain:
        # Gamma is typically given per share, so multiply by 100 shares/contract
        gamma_per_contract = contract.gamma * 100

        if contract.type == 'call':
            total_call_gex_per_point += contract.open_interest * gamma_per_contract
        elif contract.type == 'put':
            total_put_gex_per_point += contract.open_interest * gamma_per_contract

    # GEX is the change in delta per 1 point move.
    # To get the notional value hedged per 1 point move, multiply by spot price.
    # The formula in the prompt uses a sign convention where puts are negative.

    total_gex_notional = (total_call_gex_per_point - total_put_gex_per_point) * spot_price
```

```
    # Result is often expressed in billions of dollars.
    return total_gex_notional / 1_000_000_000 # GEX in $ Billions per 1% move
```

3.3. Critical Analysis

- **Data Quality and Scope:** The accuracy of the GEX calculation is highly sensitive to the quality and scope of the input data. It requires accurate, real-time (or end-of-day) Open Interest and calculated Gamma values. The choice of which expirations to include (e.g., all expirations vs. only the next 3 months) will significantly alter the result.
- **Assumption of Dealer Position:** The core assumption is that public open interest represents positions held against market makers, who are therefore short these options (short calls, short puts). While a strong industry heuristic, this is not universally true and can be skewed by large institutional positions that are not delta-hedged in the same way.
- **Interpretation Nuance:** GEX is an indicator, not a predictor. High positive GEX is often interpreted as a price-pinning or mean-reverting force, as market makers must sell into strength and buy into weakness to remain delta-neutral. Conversely, significant negative GEX can act as an accelerant. These interpretations are context-dependent and not guaranteed.

---

4. Client-Side Iceberg Order Implementation

4.1. Technical Deconstruction

An Iceberg Order is an algorithmic order type used to execute a large volume order without revealing the full order size to the market. It works by breaking the large "parent" order into smaller "child" limit orders, or "chunks." A new chunk is only submitted to the market after the previous one has been fully filled. This minimizes market impact and obscures the trader's full intention. The client-side implementation requires a persistent process that manages this sequential submission and monitors for fills.

4.2. Implementation Strategy

An `IcebergExecutor` class can be designed to manage the lifecycle of the order. It will hold the state, including total size, filled size, and remaining size, and will contain the logic for creating and submitting the next chunk.

Algorithm: 1. Initialize the executor with the total order size, limit price, and chunk size parameters (e.g., min/max size). 2. Start an execution loop that continues as long as `remaining_size > 0`. 3. Inside the loop, determine the size of the next chunk. To avoid detection, this size should be randomized within the configured range (e.g., 500-1,000 shares). 4. Create and submit a standard limit order for the calculated chunk size at the specified limit price. 5. The executor then enters a "waiting" state, listening for fill events from the broker that correspond to the active chunk's order ID. 6. Upon receiving a full fill notification for the chunk, update the `filled_size` and `remaining_size`, and repeat the loop from step 3. 7. The process is complete when `remaining_size` is zero.

Pseudocode for Iceberg Executor:

```
import random
import time

class IcebergExecutor:
    def __init__(self, symbol, total_qty, limit_price, side, min_chunk,
max_chunk):
        self.symbol = symbol
        self.total_qty = total_qty
        self.limit_price = limit_price
        self.side = side
        self.min_chunk = min_chunk
        self.max_chunk = max_chunk

        self.qty_filled = 0
        self.is_running = False
        self.active_chunk_order_id = None

    def start(self):
        self.is_running = True
        print("Starting Iceberg Execution...")
```

```
        self.execute_next_chunk()

    def on_broker_fill_event(self, event):
        if self.is_running and event.order_id == self.active_chunk_order_id:
            if event.status == "FILLED":
                self.qty_filled += event.filled_qty
                print(f"Chunk filled. Total filled: {self.qty_filled}/
{self.total_qty}")

                if self.qty_filled < self.total_qty:
                    # Optional: Add a random delay to obscure the pattern
                    time.sleep(random.uniform(1.0, 5.0))
                    self.execute_next_chunk()
                else:
                    self.is_running = False
                    print("Iceberg Execution Complete.")

    def execute_next_chunk(self):
        if not self.is_running:
            return

        qty_remaining = self.total_qty - self.qty_filled

        # Determine next chunk size
        chunk_size = random.randint(self.min_chunk, self.max_chunk)
        chunk_size = min(chunk_size, qty_remaining) # Ensure we don't exceed
total

        print(f"Submitting next chunk of size: {chunk_size}")

        # Create and submit the order
        chunk_order = Order(self.symbol, chunk_size, self.limit_price,
self.side)
        self.active_chunk_order_id = api.submit(chunk_order)
        # The executor now waits for the on_broker_fill_event callback
```

4.3. Critical Analysis

- **Detection Risk:** Sophisticated market participants run algorithms specifically to detect patterns like Icebergs. Constant chunk sizes or predictable timing between submissions make an Iceberg easy to spot. Randomizing both the chunk size and the time delay between submissions is crucial for stealth.
- **Market Drift:** The primary failure mode is the market price moving away from the order's limit price. If the price moves unfavorably, the Iceberg will stop executing. The executor needs a "time-in-force" or "price-chasing" policy. For example: if no fills occur for X minutes, should it cancel the entire operation, or should it re-price the next chunk to the new bid/ask? This logic is critical for completion.
- **Partial Fills of Chunks:** The simplified logic assumes full fills for each chunk. A robust implementation must handle partial fills. If a chunk is only partially filled and the price moves away, the system must decide whether to cancel the remainder of that chunk and submit a new one, or to wait for a potential fill.
- **System Resilience:** Like the Bracket Order, this is a long-running, stateful process. It must be resilient to application restarts. The state (`total_qty`, `qty_filled`) must be persisted so the algorithm can resume where it left off.